

V2 Device Security Workflow Document

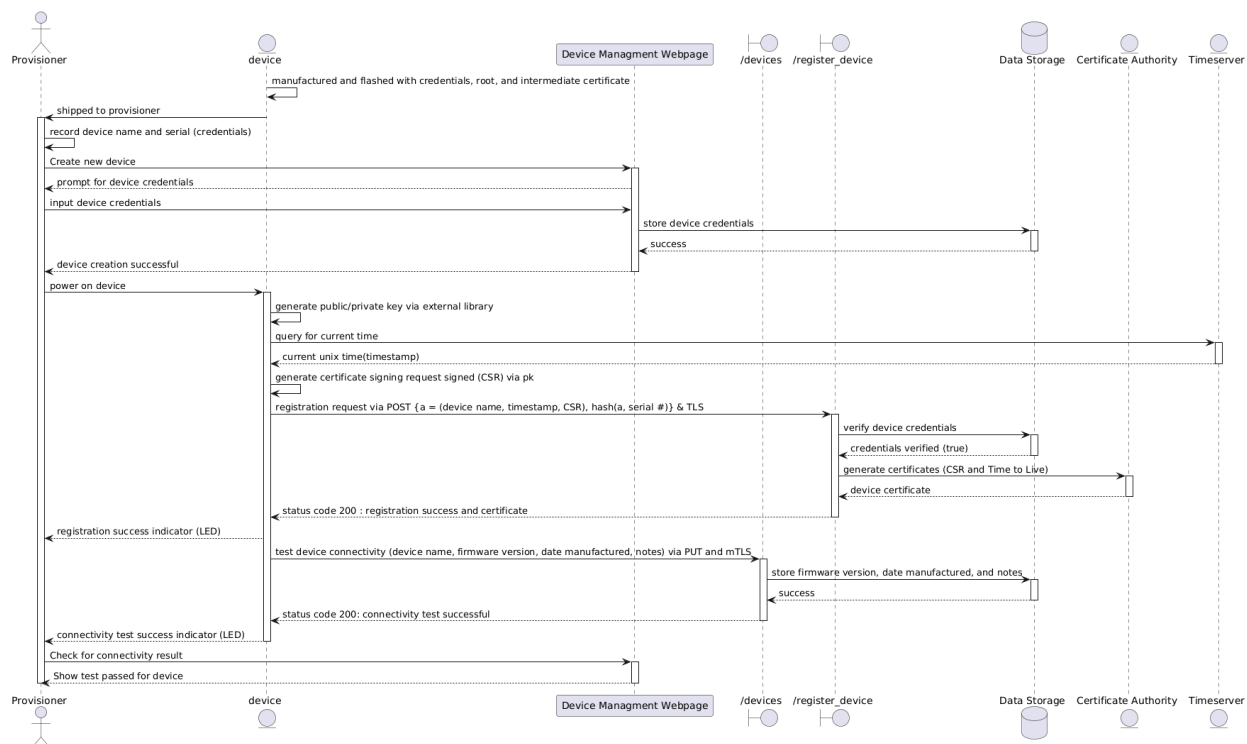


This Trailblazer's document is meant to serve as a guide for the MSD team as it pertains to properly authenticating with the server and communicating. Above is the full sequence diagram. Implementation specifics are left to the MSD team. For context, the original device securely transmitted data to the server via TLS, however its authentication with the server was insecure. Thus, we (the Trailblazers/server-side software team) have decided that only mTLS authenticated IoT devices may push data to the server. This gives technicians more control over device permissions and access to mitigation strategies should a device be compromised, guarantees that only devices manufactured and provisioned expressly by us may interact with the server, and spoofed devices cannot be used for denial of service attacks.

Certain entities such as the time server are not Trailblazer's entities, but external libraries or services that the MSD team must find and interact with. Additionally, TLS and mTLS handshake implementation is not documented in these workflows, but it is assumed that the implementor will find libraries to handle such things. mbedTLS is an example, but many modems come with such libraries already (such as the modem used by the V1 device - XBee). It's highly recommended to look at the V1 code to understand how this was implemented. More on this in the [V1.5 Patch Plan](#).

Each boundary EX: "/devices" is the actual endpoint to hit for the requests that go to them. Full urls will be added when they are set up. Additionally, wherever present, "Timestamps" are assumed to be integer unix timestamps.

First Start - Provisioning



This workflow runs through all the necessary steps in the provisioning process from manufacture to ready-for-installation. How the device receives and shows feedback to the provisioner (buttons, LEDs) are just suggestions - but interfacing with the device to set off certain functions is critical. More on this in the section: [Connectivity Test](#). It is assumed that during the manufacturing process, the device will be flashed with a unique device name and serial number that you securely generate. These **must be unique** or device connectivity will fail. Additionally, they will be flashed with the server's root and intermediate certificates. **Ensure that the modem chosen for V2 can support an intermediate certificate for maximum security - Xbee does not support intermediate certificates.** These certificates are how the device will perform the initial TLS handshake with the server, and will be provided to you. The devices must be shipped with their credentials (name and serial number, both strings) to the provisioner, who will securely record these credentials and input them into our provisioning system at [trailcount.io](#). The provisioner must have registered for an account and have administrative access, which should be requested from the project sponsor.

The step "power on device" can be exchanged for some other trigger such as receiving network connectivity, however it should be the first step a device takes. Registration with the server is required to communicate with it via mTLS. "pk" in the CSR generation step refers to the private key generated earlier. This should be generated with any secure key-pair generation library. During the registration request phase, it sends four items in the request body. The device name (string), the timestamp (integer) from the time server, the CSR, as well as all three of those items hashed with SHA512 against the device serial number (encoded to utf8) in the following format: `f"{device_name}:{timestamp}:{csr}".encode(utf8)`. If this is successful, it will return a 200 status code as well as the certificate the device should use for mTLS. Immediately after a successful registration, the device should test its connectivity to ensure there are no problems. The results of this test can be found on the device management web page, but it is recommended that an easy indicator be added to the device for an installer to use when on the road to easily perform sanity checks during installation.

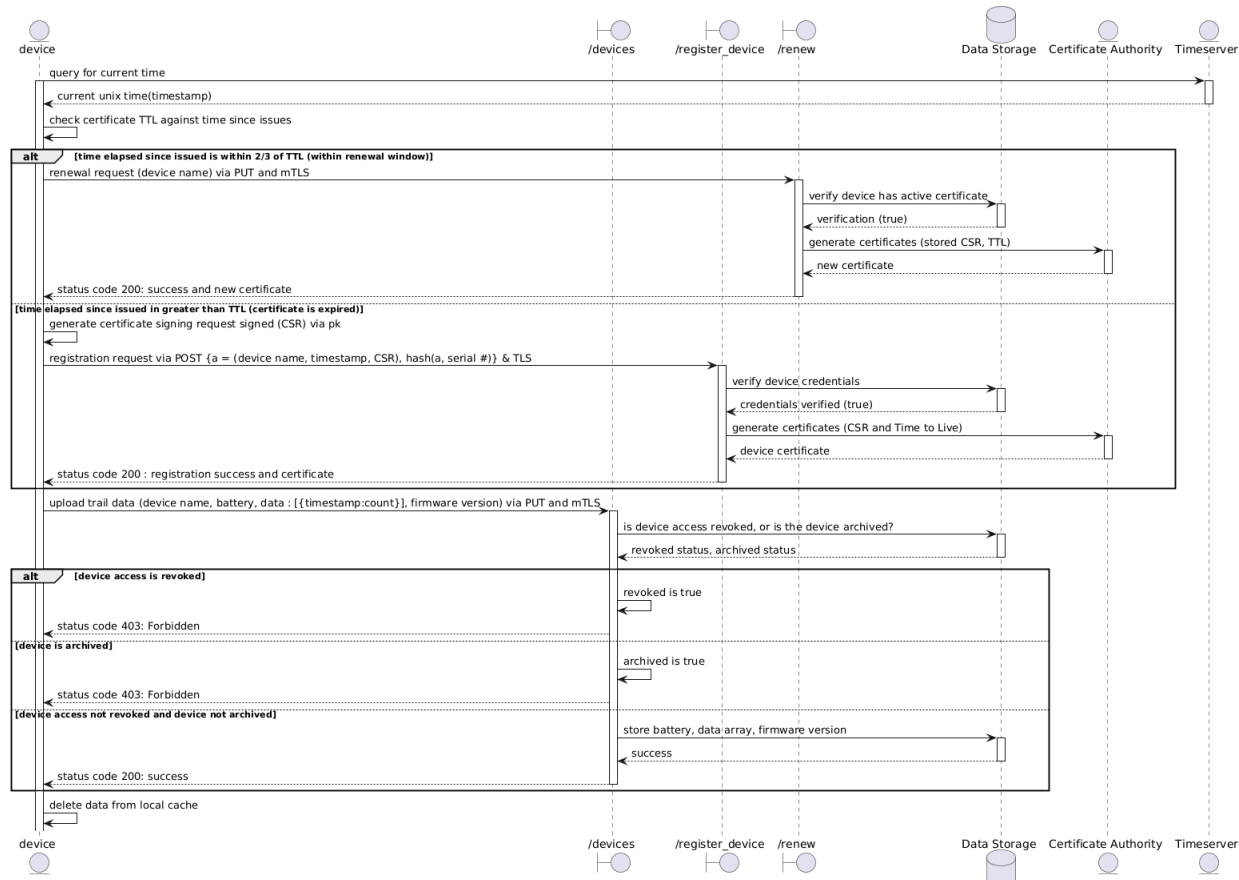
Connectivity Test



The connectivity test function is for a provisioner or installer to check that the device is connected without transmitting trail data. The trigger is suggested as a button press, but could also be something like a command passed through USART. We recommended that the trigger be kept simple and nontechnical wherever possible. If no indicator is added to the device, the

result of the connectivity test can be viewed at the device management page on the trailcount website.

Data Upload



The data upload procedure was historically triggered every night at midnight. This detail may be changed however the MSD team would like, the server is configured to handle any data coming in at any time. In fact, it's recommended for scalability and load sake that the data be transmitted at a random time around midnight (or whatever else) rather than exactly at midnight, as many (500+) devices transmitting a full day of trail data all at the exact same time is an own-goal denial of service attack. During the upload procedure, there is also the certificate renewal procedure. Certificates have a certain Time-To-Live (TTL) that must be checked during the upload procedure or whenever else the MSD team finds prudent to do so. The certificate renewal window is the last third of its TTL. For example, if the TTL is 30 days, the renewal window is the last 10 days. If the certificate is within that window, it may choose to renew its certificate through a simple request to the renewal endpoint to receive a new certificate. If the renewal window has passed, and the certificate has expired, it must register with the server again, following the same registration procedure it followed on first start up.

After this procedure, the device may upload its data. This body of the request contains the device name (string), its current battery (integer), an array of key:value pairs where a unix

timestamp (integer) is the key, and the value is the count (integer) at that timestamp, and the firmware version (string). The expected format for the data array is that there will be 24 unix timestamps corresponding to each hour in the day. The count will be how many sensor activations occurred during that hour. This is an optimization over how the data was historically transmitted.

When the server receives an upload request, it will check if the device is archived or its access revoked. Archiving a device and revoking its access are functions that may be used by an administrator from the device management page. If a device is archived or its access revoked, the upload request will return a 403 status code. If the device's upload request is successful and the data was saved to the server, it will return a 200 status code. No matter the status code, device behavior remains the same. It should delete its current data and continue saving new trail data before attempting to upload on the next upload trigger.

V1.5 Patch Plan

The patch should be very short, without needing invasive firmware updates. This patch is meant to change as little of V1 as possible and still secure communication between the device and server. This is done by manually pre-generating the certificates and uploading them to the already existing modem. The step by step guide is below:

1. Generate device public and private key.
2. Generate device certificate from root certificate.
3. Upload to Xbee:
 - a. Root & Device certificate via `ATFS PUT`
 - b. Device private key via `ATFS XPUT`
4. Configure by setting `AT$0` to:
`<root-cert-filename>.pem;<device-cert-filename>.pem;<private-key-filename>.pem`
5. Ensure that the remote host has been correctly configured to point to the Trailblazers V2 PROD API (request from Trailblazer's Team).
6. Ensure that the remote port is still correctly configured to use port `443`.
7. Reboot device and trigger immediate upload to check that the device is sending data to the server.

By manually pregenerating the certificates and nothing else, once those certificates expire, that is the end of the device's lifetime unless this process is followed again.

Resources used to develop patch:

1. Xbee Modem [User Guide](#) pages 189-190 and 258-259
2. V1 Firmware Repo (request access from sponsor)
3. Xbee Modem ANSIC [library](#)